Normalization

Normalization is a series of steps followed to obtain a Database design that allows efficient access and storage of data.
With Normalization we can –

- Eliminate data redundancy(storing the same data in more than one table)
- Ensure Data Dependencies(only storing related data in a table)
- Ensure Data Consistency
- Reduces the amount of space a database consumes
- Ensure that data is logically stored.

Normal Forms

The database community has developed a series of guidelines for ensuring that databases are normalized. The below are the normal forms we can apply to normalize the tables

- First Normal Form
- Second Normal Form
- Third Normal Form
- Boyce Codd Normal Form
- Fourth Normal Form
- Fifth Normal Form

Demonstration of Normalization with an Example
Consider student table containing student details along with his/her semester subjects as shown below.

StudentId	StudentName	Year	Semester	UnitCode	UnitName
0023765	John Doe	2009	2	UG45783	Advanced Database
0023765	John Doe	2009	2	UG45832	Network System
0023765	John Doe	2009	2	UG45734	Multi-User Operating System
0035643	Ann Smith	2009	2	UG45832	Network System
0035643	Ann Smith	2009	2	UG45951	Project
0061234	Peter Wolf	2009	2	UG45783	Advanced Database

First Normal Form

To say a table to be in First normal form , the below two rules needs to be satisfied

1. The table should be atomic - Each Column for a respective row should be atomic in the table"

2. Repeating groups of data representing a single entity should be minimized

Let us consider the above table - Which is having "Repeating groups of data" for the records of students named Ann Smith and John Doe.

As shown in the highlighted boxes, the data in some columns is repeating because of values to be accommodated in other columns.

Any way the table maintains atomicity as each column / attribute consists of single value .

To convert the above table into First normal form, We need to avoid the repeating groups. By splitting this table into two tables as Shown below, that has been achieved.

Following are the two tables Student and Student_Unit respectively.
Student Table:

StudentId	StudentName	Year	Semester
0023765	John Doe	2009	2
0035643	AnnSmith	2009	2
0061234	Peter Wofl	2009	2

Student_Unit Table:

StudentId	UnitCode	UnitName
0023765	UG45783	Advanced Database
0023765	UG45832	Network System
0023765	UG45734	Multi-User Operating System
0035643	UG45832	Network System
0035643	UG45951	Project
0061234	UG45783	Advanced Database

To identify each row uniquely , there is a primary key for each table as below

StudentId for Student table

StudentId,UnitCode (composit primary key) for Student_Unit table.

In the first table student id is primary key where as in second table the combination of student id and unit code is defined as primary key.

Second Normal Form

Let us consider the table , which is in first normal form . Let us consider the following Student_Unit table .

The table is in 1NF but still we can see redundant data in the second and third columns for the first and the last rows of the table. This redundancy has taken place because of the partial dependency that exists in the table.

The primary key of this table is a composite primary key ,consisting of columns **StudentId and UnitCode.** (Combination of these two attributes). But we can see that , the non key attribute **UnitName** can be determined by the **UnitCode** itself. That means, a part of the primary key can determine a non-key attribute.

Since, the table has partial dependency, table is not in 2NF.

What does second normal form says?

" Every non key attribute of a table should fully depend on Primary key, but not on part of the Primary Key"

To make the table to be in Second Normal Form, we have to remove the partial dependency. Hence we have to divide the table into two tables, in such a way that , each non key attribute depends fully on primary key, but not on part of Primary key

If we notice the above tables, all Non Key attributes (UnitCode in first table and Unit Name in Second table) are fully depends on Primary key

Third Normal Form

Let us consider the below table

UnitCode	UnitName	CourseCode	CourseName
UC1	Advance DB	C2009	Computing
UC2	Networking Sys	C2009	Computing
UC3	Project	B2009	Business and Computing

A table is said to be in third normal form if it satisfies the following conditions:

- It is in second normal form
- There is no transitive functional dependency

But the above table is not in Second Normal form and as well as there is a transitive dependency between Course Name ->CourseCode->UnitCode.

Which leads data inconsistency while performing DML operations on the above table, So let us divide the above table into the below tables in such a way that the resultant tables are in second normal form and as well as in third Normal Form

UnitCode	UnitName	CourseCode
UC1	Advance DB	C2009
UC2	Networking Sys	C2009
UC3	Project	B2009

CourseCode	CourseName
C2009	Computing
C2009	Computing
B2009	Business and Computing

Boyce Codd Normal Form

A relational schema S is considered to be in **Boyce–Codd normal form (BCNF)** if, for every dependencies X → Y, one of the following conditions holds true:

- X → Y is a trivial functional dependency (i.e., Y is a subset of X)
- X is a super key for schema S

So the Boyce-Codd normal form is described as "*Each attribute must represent a fact about the key, the whole key, and nothing but the key ITSELF*."

How to Decompose

To convert a table to BCNF, you must split the table using these two steps.

1. Find a nontrivial functional dependency X → Y, which violates the BCNF condition (where the X is not a super key)
2. Then Split your table in two tables:

-one with attributes XY , ie Key Set and

One with attributes of X together with the remaining attributes from the original relation

Keep **repeating** the above decomposition process until all of your tables are in BCNF.

Example

Let's take a look at this table, which is not in BCNF.

Writer	Location	Book Title	Total Page Count
W1	Hyderabad	TITLE1	200
W2	Banglore	TITLE2	100
W3	Chennai	TITLE3	300

The nontrivial functional dependencies in the table , which are violating BCNF are

Writer -> Location

BookTitle ->Total Page Count

Writer and book title together acts as a key here

Let us split into multiple tables to maintain trivial dependencies

Writer	Location
W1	Hyderabad
W2	Banglore
W3	Chennai

Book title	Total Page Count
TITLE1	200
TITLE2	100
TITLE3	300

Writer	Book title
W1	TITLE1
W2	TITLE2
W3	TITLE3

None of the above functional dependencies are not violating the BCNF Rules,

The key for the first table is **Writer** ,The key for the second table is **Book title**.
The key for the third table is **Writer** and **Book title**, together.

So the table is in BCNF form.